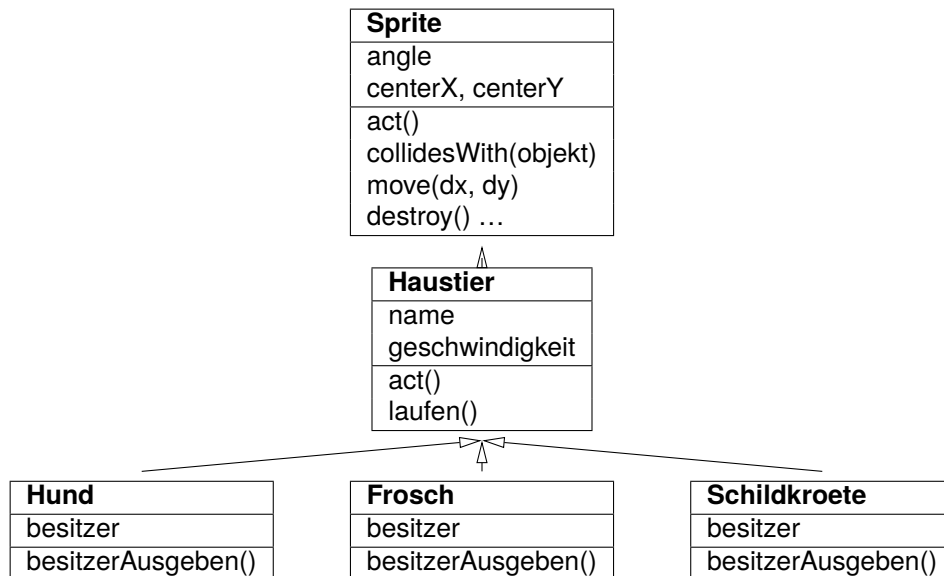


2.3 Polymorphismus



Beschreibe das Klassendiagramm und erkläre, warum die folgenden Zuweisungen erlaubt sind. Achte auf die Vererbungsbeziehungen (Ober- und Unterklassen).

```
Haustier tier1 = new Hund();
Haustier tier2 = new Frosch();
Haustier tier3 = new Schildkroete();
```

Hund, Frosch und Schildkroete sind Unterklassen von Haustier (das von Sprite erbt). Jedes ihrer Objekte ist auch ein Haustier und passt deshalb in eine Variable vom Typ Haustier.

Aufgabe 1 Projekt Haustier-Rennen

a) Ausführen. b) Deinen Namen als Besitzer eintragen. c) `h.besitzerAusgeben()` aufrufen. Was meldet der Compiler? Welche Methoden darf er für `h` überhaupt zulassen?

Fehler: `besitzerAusgeben()` gibt es in der Klasse Haustier nicht. Er lässt nur Methoden von Haustier und Sprite zu, z. B. `act()`, `laufen()`, `move(...)`.

Polymorphie bei höheren Datentypen

Speichert man ein Objekt der Unterklasse in einer Variablen vom Datentyp der Oberklasse, so ist es nicht mehr möglich, Attribute und Methoden zu nutzen, die nur auf der Unterklasse definiert sind. Stattdessen wird das Objekt der Unterklasse wie sein Behälter (mit dem Datentyp der Oberklasse) behandelt. Hier hat also eine implizite Typumwandlung stattgefunden. Diese Art der Typumwandlung, bei der ein Objekt gleichzeitig mehrere Typen hat, bezeichnet man als Polymorphie (griechisch: Vielgestaltigkeit).

Aufgabe 2 Projekt Haustier-Rennen

a) `besitzerAusgeben()` in `Haustier` ergänzen. **b)** Teil 2 im Hauptprogramm einkommentieren. Vorhersage der drei Ausgaben:

Rex gehört Ute

Quaki gehört (dein Name)

Emma gehört Sirona

c) Starte. Warum erscheint nicht dreimal „Ein Haustier ohne Besitzer“?

Die Unterklassen überschreiben `besitzerAusgeben()`. Erst beim Ausführen zählt das tatsächliche Objekt (Hund, Frosch, Schildkröte), und dessen Methode läuft.

Dynamische Bindung

Wird ein Objekt der Unterklasse im Behälter der Oberklasse gespeichert, so sind nur die Methoden der Oberklasse verfügbar. Diese können jedoch von den Unterklassen überschrieben werden. Deshalb muss *während der Ausführung* geprüft werden, ob die Methode überschrieben ist. Ist das der Fall, wird die Methode der Unterklasse ermittelt und ausgeführt. Diese Art der Vermittlung nennt man dynamische Bindung. Um im Programmcode klarzumachen, dass eine Methode überschrieben wird, nutzt man die Annotation `@Override` in der Zeile über der Methode.